There's a server-side component to this command that is another method in the concert channel:

chapter_11/02/app/channels/concert_channel.rb

```
def removed_from_cart(data)
  cart = ShoppingCart.find_or_create_by(user_id: data["userId"])
  cart.clear(
    concert_id: data["concertId"],
    tickets: data["tickets"]
  )
  result = Ticket.data_for_concert(data["concertId"])
  ActionCable.server.broadcast("concert_#{data["concertId"]}", result)
end
```

And there is a small `clear` method in the shopping cart:

chapter_11/02/app/models/shopping_cart.rb

```
def clear(concert_id:, tickets:)
  tickets.each do |ticket|
    db_ticket = Ticket.find_by(
      row: ticket["row"], number: ticket["number"], concert_id: concert_id
    )
    db_ticket&.update(status: :unsold)
  end
end
```

So far, we've been using reducers in the single-threaded, synchronous JavaScript world. Specifically, this means that all the state changes that go through the reducer happen in order, and we know when we call `dispatch` exactly what order state changes will happen and when state changes are done.

Asynchronous changes break that assumption. We don't know when state changes are going to happen once we introduce asynchronous activities like talking to the server or receiving data from ActionCable. This is why we had to do our asynchronous fetch data call outside the reducer.

The key here is we need to keep asynchronous actions away from the reducer, as we've been doing by putting them in the components. The incoming subscription